ESP32 Firmware Flashing Tutorial

1) Create and activate Conda environment

```
conda create -n esp32 python=3.11 -y
conda activate esp32
```

💡 **Note:** To process and save CSI data in Python, make sure you have the required packages installed:

```
pip install pyserial numpy h5py
```

---

2) Get and install ESP-IDF

ESP-IDF is Espressif official development framework.

```
# Choose any install path, here we use "esp32" folder
mkdir esp32 && cd esp32
git clone --recursive https://github.com/espressif/esp-idf.git
cd esp-idf

# Install tools for ESP32 (WROOM series)
./install.sh esp32
```

---

3) Load environment variables (every time before use)

```
cd esp-idf
source ./export.sh
```

---

4) Verify toolchain

```
idf.py --version
```

You should see the ESP-IDF version printed.

---

5) Download ESP-CSI example

```
cd ..
git clone https://github.com/espressif/esp-csi.git
cd esp-csi/examples/get-started/csi_recv_router/
```

---

6) Select target chip (only first time or when switching)

```
idf.py set-target esp32
```

---

7) Configure, build, and flash

```
# Open configuration menu
idf.py menuconfig
```

```
(Top)
                                    Espressif IoT Development Framework Configuration
    Build type  --->
    Bootloader config  --->
    Security features  --->
    Application manager  --->
    Serial flasher config  --->
    Partition Table  --->
    Example Connection Configuration  --->
    Compiler options  --->
    Component config  --->
[ ] Make experimental features visible



[Space/Enter] Toggle/enter  [ESC] Leave menu           [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

```
(Top) → Example Connection Configuration
                                    Espressif IoT Development Framework Configuration
[*] connect using WiFi interface
[ ]     Get ssid and password from stdin
[*]     Provide wifi connect commands
(myssid) WiFi SSID
(mypassword) WiFi Password
(6)     Maximum retry
        WiFi Scan Method (All Channel)  --->
        WiFi Scan threshold  --->
        WiFi Connect AP Sort Method (Signal)  --->
[ ] connect using Ethernet interface
[ ] connect using Point to Point interface
[*] Obtain IPv6 address
    ↓↓↓↓↓↓↓↓↓↓↓↓↓
[Space/Enter] Toggle/enter  [ESC] Leave menu           [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

```
# Build project
idf.py build

# Flash firmware to the ESP32 board
idf.py flash
```

---

8) Open serial monitor

```
idf.py monitor
```

You should now see CSI output from your ESP32 in the terminal.

Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H

```
I (4390) esp_netif_handlers: example_netif_sta ip: 172.20.10.12, mask: 255.255.255.240, gw: 172.20.10.1
I (4391) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.12
I (4392) example_common: Connected to example_netif_sta
I (4392) example_common: - IPv4 address: 172.20.10.12,
I (4393) example_common: - IPv6 address: fe80:0000:0000:0000:fee8:c0ff:fe76:31d8, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (4395) csi_recv_router: got ip:172.20.10.12, gw: 172.20.10.1
I (4398) main_task: Returned from app_main()
I (4432) csi_recv_router: ================ CSI RECV ================
type,id,mac,rssi,rate,sig_mode,mcs,bandwidth,smoothing,not_sounding,aggregation,stbc,fec_coding,sgi,noise_floor,ampdu_cnt,channel,secondary_channe
l,local_timestamp,ant,sig_len,rx_state,len,first_word,data
CSI_DATA,0,2e:54:1a:3f:d3:09,-74,11,1,0,0,1,1,0,0,0,0,-96,0,6,0,3041445,0,83,0,128,1,"[83,-80,4,0,13,9,17,10,21,10,24,11,27,11,28,10,27,9,24,6,21,
5,18,4,16,5,12,4,8,3,3,1,-1,-1,-6,-2,-10,-4,-12,-6,-14,-8,-16,-11,-16,-13,-16,-16,-14,-17,-11,-17,-9,-16,-2,-4,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
```

9) Run the CSI data collection script

Once the firmware is running and the ESP32 is outputting CSI data, go to the folder where `csi_data_collection.py` is located.

Then run the script:

```
python csi_data_collection.py
```

- This script will connect to the serial port, parse the CSI data, and **store it in an HDF5 file**.
- The `.h5` file will be created automatically (named with a timestamp, e.g. `csi_2025-08-24_14-30-10.h5`).
- Inside the file:
  - **`/csi` dataset** → contains CSI values (complex numbers)
  - **`/meta` dataset** → contains metadata (RSSI, channel, timestamp, etc.)

```
(esp32) zhongqinwang@Zhongqin-Wang ESP32 % python csi_data_collection.py
open success
[HDF5] Creating file: csi_2025-08-24_00-59-26.h5
[Frame] time=2025-08-24 00:59:28.876 rssi=-69 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.885 rssi=-70 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.935 rssi=-69 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.962 rssi=-70 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.969 rssi=-70 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.978 rssi=-70 dBm ch=6 len=128 vecN=108
[Frame] time=2025-08-24 00:59:28.987 rssi=-67 dBm ch=6 len=128 vecN=108
```